

Ejercicio 6. Propuestas de mejora

Proyecto: Data Warehouse dimensional para el monitoreo geoespacial de obras públicas municipales

Asignatura: Base de Datos **Equipo:** (nombres de los integrantes) **Fecha:** (fecha de entrega)

Resumen de propuestas

N.º	Propuesta	Técnica de IA/ML	Dificultad	Responsable
1	Agrupar por tema las propuestas de la gente y avisar cuando se repiten	Sí	Media	(integrante)
2	Guardar el historial de cada obra: estados, presupuesto y constructora	No	Media	(integrante)
3	Proteger los datos personales y llevar un registro de quién entra al sistema	No	Alta	(integrante)
4	Avisar qué obras podrían tener problemas de costo o de avance	Sí	Media	(integrante)
5	Revisar que las fotos y archivos guardados coincidan con la base de datos	No	Baja	(integrante)
6	Guardar la ubicación real de las obras y comprobar que las fotos se tomaron ahí	No	Media	(integrante)

Cada propuesta se describe con cinco apartados: (1) problema que atiende, (2) uso desde la perspectiva del usuario, (3) cambios en el modelo de datos, (4) fundamento y (5) dificultad estimada.

Propuesta 1. Agrupar por tema las propuestas de la gente y avisar cuando se repiten

1. Problema que atiende

En `propuestas_obras` la gente escribe con sus propias palabras (título, descripción, beneficios para la comunidad), y con muchas propuestas es imposible leerlas todas. Si varias personas piden lo mismo con textos distintos, los votos se reparten y las necesidades menos populares se pierden (el artículo lo menciona como limitación en el punto 2 de la Sección 8.2). Además, el campo `region` de esa tabla es texto libre y no está conectado con la tabla `region`.

2. Uso desde la perspectiva del usuario

El planeador entra a “Demandas por tema” y ve las propuestas ya agrupadas (agua, caminos, escuelas...), con cuántas propuestas y votos tiene cada tema por región. Cuando un ciudadano escribe una propuesta nueva, el sistema le avisa si ya existe una muy parecida y le deja apoyar esa en lugar de repetirla.

3. Cambios en el modelo de datos

Entidad	Cambio	Descripción	Campos
tema_propuesta	Nueva	Guarda los temas.	id_tema , nombre , palabras_clave , version_modelo
propuestas_obras	Modificada	Se agregan campos para el tema y la propuesta parecida. Además, region deja de ser texto libre y pasa a conectarse con la tabla region .	Nuevos: id_tema , puntaje_pertenencia (qué tanto encaja en el tema), id_propuesta_similar , similitud
propuesta_embedding	Nueva	Guarda una representación numérica del texto de cada propuesta, que sirve para comparar su significado.	id_propuesta , vector , modelo

Relaciones: un tema agrupa muchas propuestas; cada propuesta tiene su representación numérica; una propuesta puede apuntar a otra parecida.

4. Fundamento

- Una limitación del artículo: la votación simple puede no representar bien a las minorías.
- Una necesidad identificada por el equipo: los textos ya se guardan, pero nadie los aprovecha, y el artículo no usa ningún análisis de texto.
- **Parte de IA:** un modelo convierte cada texto en números según su significado y agrupa los que hablan de lo mismo. Se justifica con los datos que ya existen en `propuestas_obras` y `votos_propuestas` .

5. Dificultad estimada: Media

Existen herramientas ya hechas para el análisis de texto; lo difícil es comprobar que los grupos tengan sentido y armar la pantalla de revisión.

Propuesta 2. Guardar el historial de cada obra: estados, presupuesto y constructora

1. Problema que atiende

En **obra**, el campo **estado** solo puede decir sí o no, y **etapa** es un número sin significado definido. Pero el mapa del artículo necesita tres estados: terminada, en proceso y retrasada. Además, cuando algo cambia (el presupuesto, la constructora o el estado), el dato anterior se pierde. Esa memoria de lo que pasó es justo lo que el artículo vende como su mayor ventaja para auditar (Sección 4.2).

2. Uso desde la perspectiva del usuario

El auditor abre una obra y ve una línea de tiempo: cuándo cambió de etapa, quién cambió el presupuesto, de cuánto a cuánto y por qué motivo. Puede responder preguntas como "¿cuánto presupuesto tenía esta obra el día que se autorizó el pago?". El supervisor ve si la obra está terminada, en proceso o retrasada, en lugar de un simple sí o no.

3. Cambios en el modelo de datos

Entidad	Cambio	Descripción	Campos
estado_obra	Nueva	Lista los estados posibles.	id_estado , nombre , descripcion
historial_estado_obra	Nueva	Guarda cada cambio de estado.	id , id_obra , id_estado , fecha_inicio_vigencia , fecha_fin_vigencia , es_vigente , codigo_personal , motivo
obra	Modificada	El campo estado (sí/no) se cambia por id_estado_actual , conectado con estado_obra .	Nuevo: id_estado_actual
presupuesto_obra_version	Nueva	Guarda cada versión del presupuesto.	id_version , id_presupuesto , presupuesto_total , vigente_desde , vigente_hasta , es_vigente , motivo_modificacion , codigo_personal
obra_constructora_historial	Nueva	Guarda qué constructora tuvo la obra y cuándo.	id , id_obra , id_constructora , vigente_desde , vigente_hasta , motivo_cambio

Relaciones: una obra tiene muchos cambios de estado y muchas constructoras a lo largo del tiempo; un presupuesto tiene muchas versiones.

4. Fundamento

La Sección 4.2 del artículo explica estos casos (cambio de presupuesto y cambio de constructora). El artículo logra el historial en su almacén de datos, pero en nuestro modelo no se conserva nada: es algo que encontramos al revisar el diseño.

5. Dificultad estimada: Media

Hay que pasar los datos actuales al nuevo formato y programar que, cada vez que algo cambie, se cierre la versión vieja y se abra la nueva.

Propuesta 3. Proteger los datos personales y llevar un registro de quién entra al sistema

1. Problema que atiende

La CURP de los pobladores se guarda tal cual, sin protección, en `pobladores.curp`. Además, las contraseñas están repartidas en dos tablas (`pobladores` y `personal`) y no queda registro de quién consultó o cambió información. El artículo reconoce que el rol del personal se manda en un encabezado y que la sesión del ciudadano nunca caduca (Secciones 3.2 y 8.2, punto 7). También explica que guardar identificadores como la CURP es un riesgo para la privacidad (Sección 4.2).

2. Uso desde la perspectiva del usuario

Cada persona inicia sesión y recibe un acceso que vence después de un tiempo; el sistema comprueba su rol cada vez. El director entra a una pantalla de "Accesos" para ver quién consultó o modificó qué, y cuándo. El ciudadano ve en qué dispositivos tiene la sesión abierta y puede cerrarlas. La CURP ya no aparece completa en pantalla.

3. Cambios en el modelo de datos

Entidad	Cambio	Descripción	Campos
<code>pobladores</code>	Modificada	<code>curp</code> se reemplaza por <code>curp_hash</code> (una versión protegida que sirve para verificar que no se repita) y, si hace falta poder leerla, <code>curp_cifrada</code> .	Nuevos: <code>curp_hash</code> , <code>curp_cifrada</code>
<code>sesion</code>	Nueva	Guarda cada inicio de sesión.	<code>id_sesion</code> , <code>tipo_usuario</code> , <code>id_poblador</code> , <code>codigo_personal</code> , <code>token_hash</code> , <code>fecha_emision</code> , <code>fecha_expiracion</code> , <code>fecha_revocacion</code> , <code>ip</code>
<code>bitacora_acceso</code>	Nueva	Guarda cada acción hecha dentro de una sesión.	<code>id</code> , <code>id_sesion</code> , <code>endpoint</code> , <code>metodo</code> , <code>tabla_afectada</code> , <code>id_registro</code> , <code>resultado</code> , <code>fecha</code>
<code>permiso_rol</code>	Nueva	Define qué puede hacer cada rol.	<code>rol</code> , <code>accion</code>

Relaciones: un poblador o un trabajador puede tener muchas sesiones; cada sesión tiene muchas acciones registradas.

4. Fundamento

La limitación 7 y la Sección 3.2 del artículo (control de acceso sencillo), más un hallazgo en nuestro modelo: la CURP está sin proteger y las contraseñas están en dos tablas distintas.

5. Dificultad estimada: Alta

El cambio en la base de datos es sencillo, pero cambiar la forma de iniciar sesión afecta a todas las partes de la API.

Propuesta 4. Avisar qué obras podrían tener problemas de costo o de avance

1. Problema que atiende

El artículo reconoce que su sistema casi no detecta dos problemas importantes: obras que cuestan más de lo normal (solo detecta 9 de cada 100) y obras a las que se les paga por adelantado más de lo que han avanzado (solo detecta 6 de cada 100). Esto pasa porque compara todas las obras entre sí, como si una escuela y un camino debieran costar lo mismo.

2. Uso desde la perspectiva del usuario

El planeador o el director entra a una pantalla llamada "Obras en riesgo" y ve cada obra con una calificación y una explicación sencilla, por ejemplo: "ya se pagó el 75 % pero solo se ha construido el 30 %" o "cuesta el doble de lo que suele costar una obra de este tipo". Después puede marcar cada aviso como "sí era un problema" o "no lo era". Con esas respuestas, el sistema aprende a equivocarse menos.

3. Cambios en el modelo de datos

Entidad	Cambio	Descripción	Campos
tipo_obra	Nueva	Guarda los tipos de obra (escuela, camino, agua potable...).	id_tipo , nombre
obra	Modificada	Se agregan el tipo y el tamaño de la obra.	Nuevos: id_tipo (para saber de qué tipo es), metas_o_superficie (qué tan grande es)
riesgo_obra	Nueva	Guarda la calificación de cada obra en cada informe.	id_riesgo , id_obra , id_informe , id_modelo , puntaje_riesgo , costo_esperado , brecha_avance , causa_probable , fecha_calculo
retroalimentacion_alerta	Nueva	Guarda la respuesta de la persona.	id , id_riesgo , codigo_personal , veredicto , comentario , fecha

Relaciones: una obra tiene muchas calificaciones; un informe, una o varias; cada calificación puede recibir varias respuestas.

4. Fundamento

- Calcular el costo esperado por tipo de obra, que es una de las líneas de trabajo futuro del artículo (punto 5).
- El sistema ya guarda los datos que hacen falta: el avance físico y el avance del presupuesto.
- **Parte de IA:** un modelo aprendería cuánto debería costar una obra según su tipo y tamaño, y avisaría cuando el costo real se aleje mucho de eso. También mediría la diferencia entre lo pagado y lo construido.

5. Dificultad estimada: Media

Los datos ya existen; lo que lleva trabajo es entrenar el modelo, probarlo y crear las tablas nuevas.

Propuesta 5. Revisar que las fotos y archivos guardados coincidan con la base de datos

1. Problema que atiende

Cuando un supervisor sube una foto, ésta se guarda en un lugar (Cloudflare R2) y sus datos en otro (la base de datos). Si algo falla entre un paso y otro, puede quedar una foto sin registro, o un registro

que apunta a una foto que ya no existe. El artículo (Sección 5) admite que esta revisión no está hecha.

2. Uso desde la perspectiva del usuario

El director abre una pantalla llamada "Integridad de evidencias" y ve tres listas: fotos que están guardadas pero sin registro, registros sin foto, y fotos repetidas. Para cada caso decide si lo borra, lo asocia a un informe o lo marca como revisado.

3. Cambios en el modelo de datos

Entidad	Cambio	Descripción	Campos
<code>imagenes_informe</code>	Modificada	Se agregan campos para verificar cada foto.	Nuevos: <code>hash_sha256</code> * (una huella única de la foto, sirve para detectar repetidas), <code>estado_integridad</code> (válida, huérfana, faltante o duplicada), <code>fecha_verificacion</code>
<code>revisiones</code>	Nueva	Guarda cada revisión que se hace.	<code>id_intento</code> , <code>fecha_inicio</code> , <code>fecha_fin</code> , <code>objetos_revisados</code> , <code>incidencias</code>
<code>incidencia_evidencia</code>	Nueva	Guarda cada problema encontrado y cómo se resolvió.	<code>id_incidencia</code> , <code>id_intento</code> , <code>id_imagen</code> , <code>llave_objeto</code> , <code>tipo</code> , <code>resolucion</code> , <code>codigo_personal</code> , <code>fecha_resolucion</code>

Relaciones: una revisión puede encontrar muchos problemas; una imagen puede tener cero, uno o varios problemas.

* El SHA-256 es una función criptográfica que transforma cualquier tipo de dato (texto, contraseña, archivo) en una cadena de longitud fija.

4. Fundamento

Es una limitación que el artículo declara en su Sección 5 (y que también menciona en la 3.1). El sistema ya guarda `ruta_r2`, `url_publica` y `tamaño_bytes` en `imagenes_informe`, lo que facilita comparar lo que dice la base con lo que de verdad hay guardado.

5. Dificultad estimada: Baja

Se trata de listar los archivos guardados, compararlos con la base y anotar las diferencias en dos tablas nuevas.

Propuesta 6. Guardar la ubicación real de las obras y comprobar que las fotos se tomaron ahí

1. Problema que atiende

Ninguna tabla del modelo guarda coordenadas: `region` solo tiene comunidad, barrio y colonia. El artículo pone cada obra en el mapa en su comunidad, con un pequeño desplazamiento al azar de hasta 150 metros, y reconoce que no se conoce el sitio exacto (Sección 6). Es decir, se presenta como un sistema con mapa, pero no guarda dónde están realmente las obras. Tampoco se puede comprobar si una foto se tomó en la obra.

2. Uso desde la perspectiva del usuario

El supervisor registra la ubicación real de la obra desde su celular, y cada foto de un informe queda guardada con el lugar donde se tomó. Si la foto se tomó muy lejos de la obra, el sistema la marca como “fuera de rango”. El ciudadano ve la obra en el mapa en su lugar real.

3. Cambios en el modelo de datos

Entidad	Cambio	Descripción	Campos
<code>obra</code>	Modificada	Se agrega la ubicación de la obra. Si se usa PostGIS, se puede guardar como un solo campo de punto.	Nuevos: <code>latitud</code> , <code>longitud</code> , <code>precision_ubicacion_m</code> (qué tan exacta es), <code>metodo_ubicacion</code> (GPS, manual o aproximada)
<code>imagenes_informe</code>	Modificada	Se agrega el lugar donde se tomó cada foto y su distancia a la obra.	Nuevos: <code>latitud</code> , <code>longitud</code> , <code>fecha_captura</code> , <code>distancia_a_obra_m</code> , <code>fuera_de_rango</code>

Relaciones: no cambian; los datos nuevos van en tablas que ya están conectadas.

4. Fundamento

Es una limitación que reconoce el artículo (Sección 6, el sitio exacto no está levantado) y, en nuestro modelo, no hay coordenadas en ningún lado. Además, hace que las fotos de `imagenes_informe` sirvan como evidencia comprobable.

5. Dificultad estimada: Media

Hay que activar PostGIS, leer la ubicación que trae la foto o el celular y decidir a qué distancia una foto deja de considerarse válida.